


# CSE 3421: Software Engineering

**Sajid Ahmed Chowdhury**
Lecturer, Department of Computer Science and Engineering,
**United** International **University**

# Version Control

# What is Version Control?

**Version Control** is a system that records changes to files over time, allowing multiple people to collaborate efficiently.

**Why Version Control?**

- ✅ Tracks changes in code and documentation
- ✅ Enables rollback to previous versions.
- ✅ Facilitates collaboration among developers.
- ✅ Prevents accidental data loss.
- ✅ Supports multiple versions of a project.



# Why Do We Need Git?

**Git is a distributed version control system (DVCS)** that allows multiple developers to work on a project without conflicts.

**Why Git?**

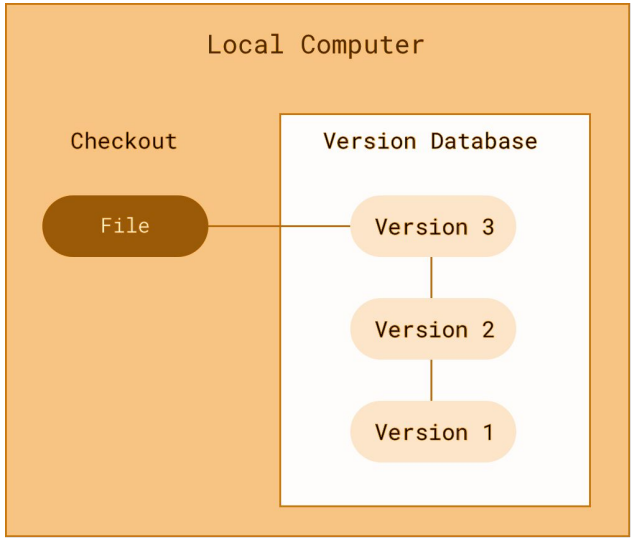
✅ Works offline – No internet required for local commits.
✅ Fast and lightweight – Handles large projects efficiently.
✅ Supports non-linear development through branches.
✅ Allows easy code collaboration via GitHub, GitLab, or Bitbucket.
✅ Prevents code conflicts with merge strategies.

# Types of Version Control System

# Local Version Control

Tracks changes **only on a single machine** (e.g., backup copies).

Example: **RCS (Revision Control System)**

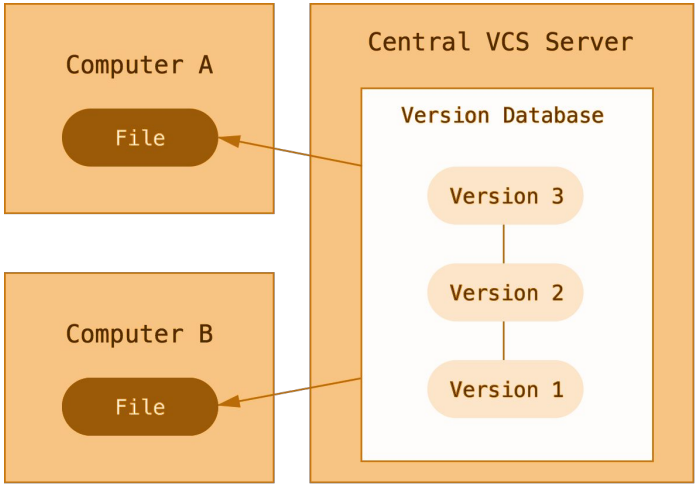


# Centralized Version Control (CVCS)

Uses a **single server** where all changes are stored.

Example: **SVN (Apache Subversion)**

**Disadvantage:** If the server fails, all history is lost.



# Centralized Version Control (CVCS)

**Single Central Repository:** All project files are stored in one central location.

**Client-Server Architecture:** Developers (clients) connect to the central server to access the codebase.

**Requires Internet Connection:** Since the repository is hosted centrally, developers must be online to make commits or retrieve updates.

**Limited Redundancy:** If the central server fails, all version history can be lost unless backups are maintained.

# Workflow of CVCS

1. **Checkout:** Developers pull a working copy of the repository.

2. **Edit:** Developers modify the code locally.

3. **Update:** Developers pull the latest changes before committing to ensure they have the most updated code.

4. **Commit:** Developers submit their changes to the central repository

5. **Conflict Resolution:** If conflicts arise, they must be resolved manually before committing.

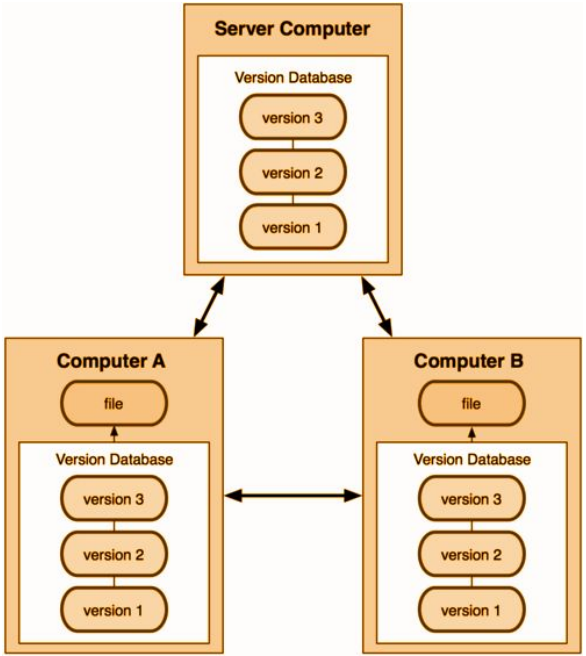
# Distributed Version Control (DVCS)

A **Distributed Version Control System (DVCS)** is a system where **every developer maintains a complete copy of the repository, including its history**.

Unlike CVCS, there's no dependency on a central server for most operations.

**Popular DVCS Tools**

Git, Mercurial, Bazaar



# Distributed Version Control (DVCS)

1. **Clone:** Developers copy the entire repository (including history) to their local system.
2. **Branch:** Developers create independent branches to isolate new features or bug fixes.
3. **Commit:** Developers make local commits without immediate server interaction.
4. **Merge:** Developers combine changes from different branches to maintain a clean project structure.
5. **Push/Pull:** Developers sync their changes with others through a shared remote repository.

# Advantages of DVCS

✅ **Offline Work:** Developers can work independently without needing constant internet access.
✅ **Faster Operations:** Local commits, branching, and history browsing are faster since no server interaction is required.
✅ **Improved Collaboration:** Developers can push and pull code directly between systems, promoting flexible teamwork.
✅ **Enhanced Data Safety:** Since every clone is a full backup, the risk of data loss is minimized.
✅ **Efficient Branching and Merging:** DVCS tools like Git excel in handling multiple feature branches with minimal conflicts.

# Disadvantages of DVCS

**Steeper Learning Curve:** DVCS tools often require more complex commands compared to CVCS.

**Potential for Duplication:** Since each clone stores the full history, large repositories may consume significant storage.

**Complex Conflict Resolution:** Merging complex changes across multiple branches can be challenging.

# DVCS vs CVCS

| Feature | DVCS (e.g., Git) | CVCS (e.g., SVN) |
|---|---|---|
| Repository Structure | **Each user has a full copy** | Single central server |
| Offline Support | **Fully functional offline** | Requires constant connection |
| Performance | **Faster for local operations** | Slower with large teams |
| Backup System | **Data redundancy via multiple copies** | Server failure risks data loss |
| Branching and Merging | **Efficient and lightweight** | More complex to manage |


git
≠
github
SOCIAL CODING

# Git vs GitHub

**Git** is a version control system that lets you manage and keep track of your source code history.

**GitHub** is a cloud-based hosting service or in another word a server.

# Git vs GitHub

| Feature | Git | Github |
|---|---|---|
| Type | **Version Control System** | Cloud-based Repository Hosting |
| Purpose | **Manages local repositories** | Hosts repositories remotely |
| Work Mode | **Works offline** | Requires internet for remote repositories |
| Collaboration | **Limited to local system** | Enables online collaboration, issue tracking |
| Example Tools | **Git CLI, Git GUI** | GitHub, GitLab, Bitbucket |

# Branching and Merging in Git




## Branching in Git

- A **branch** is an independent line of development.
- Allows working on **features separately** without affecting the main code.
- **Common Branches:**
  `main` – Production-ready code.
  `develop` – Active development.
  `feature/bugfix` – Temporary branches for new features or fixes.

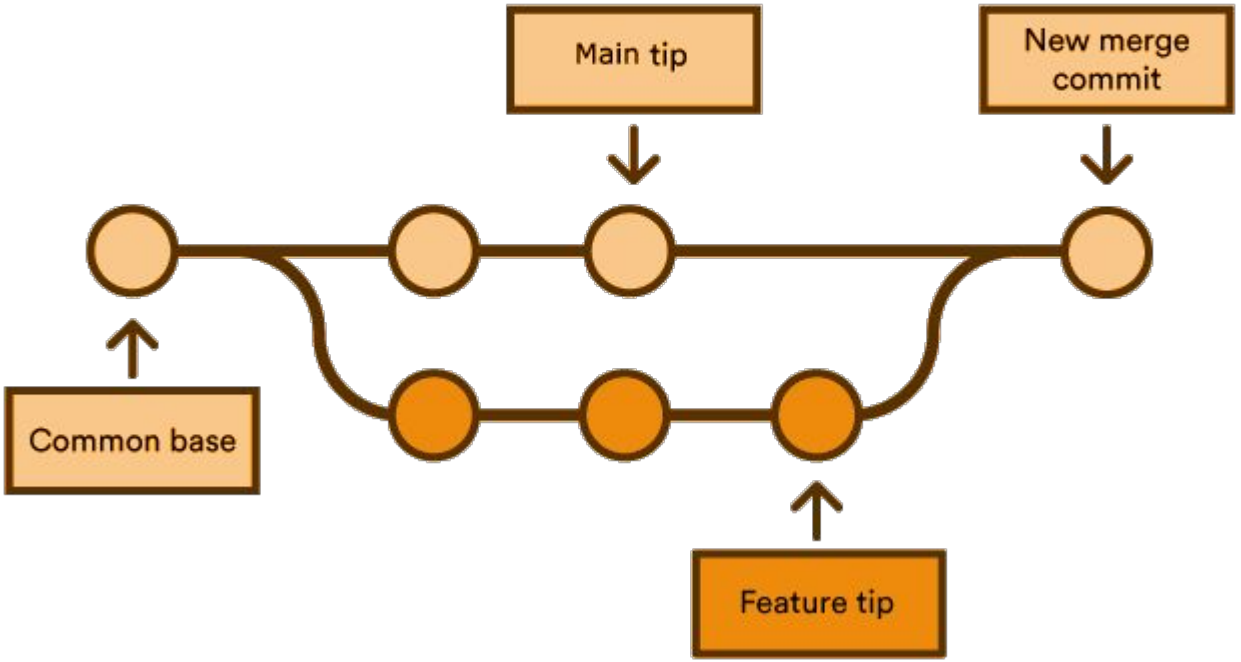
# Merging in Git

**Combines changes** from one branch into another.

**Types of Merging:**

- **Fast-forward merge:** When no other changes exist.
- **Three-way merge:** When branches have diverged.

```
git checkout main
git merge feature-new
```


Main tip
New merge commit
Common base
Feature tip

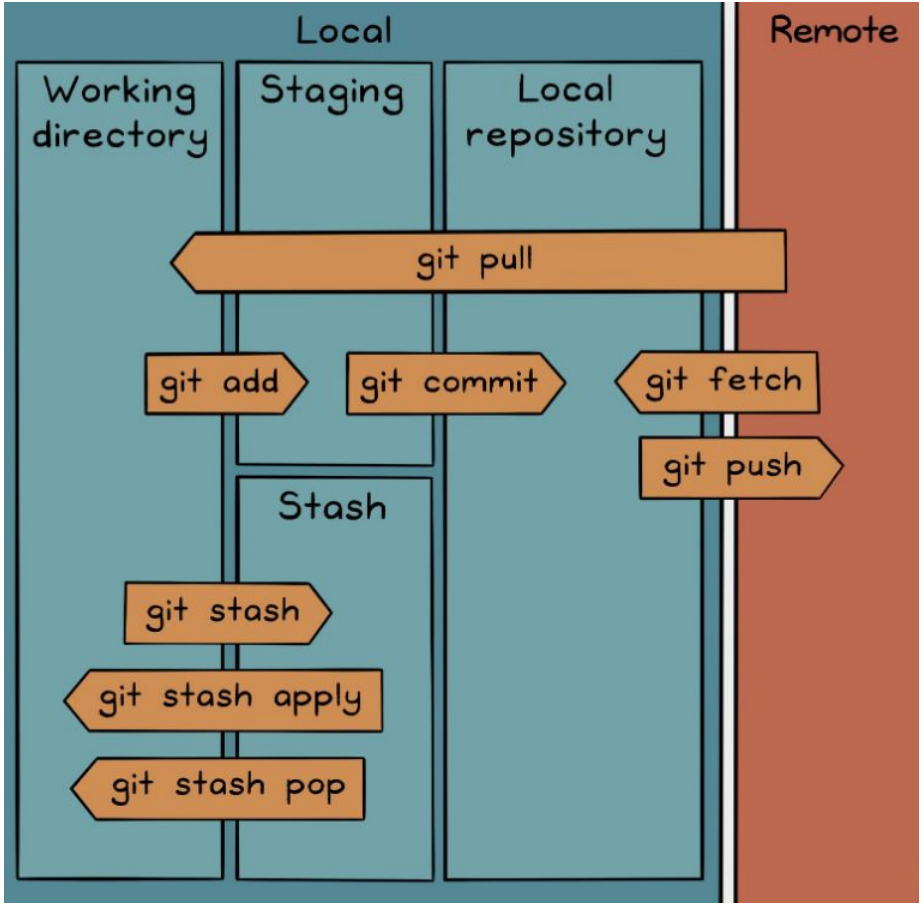
# Git Lifecycle

Working Directory

Staging Area

Local Repository

Remote Repository


Local
Remote
Working directory
Staging
Local repository
git pull
git add
git commit
git fetch
git push
Stash
git stash
git stash apply
git stash pop

*Whenever I see a door that says "push", I always pull first, to avoid conflicts.*

# Any Questions?

**Reference:**


1. Slides from Sherajul Arifin,  Lecturer, CSE, UIU
2. Git

# Thank You!